HSP pull-down data, preprocessed

Every pull-down run in this project, written out with the preprocessing already applied. Nine tables in one format, plus the 33 derived quantities the campaign modelled.

Regenerate with `python code/export_tables.py`. Nothing here is hand-edited. The whole set is also at `data/export.tar.gz` (21 MB) — `netscratch` is purged on a retention window, so copy that somewhere permanent rather than the directory.

---

1. Layout

```
data/export/
  README.md              this file
  docs/                  the written-up project, travelling with the data:
                           HSP_PULLDOWNS_OVERVIEW.{md,pdf}   dataset, goals, methods, where
                           hsp_mlp_campaign_report.{md,pdf}  the full 31-page campaign repor
                           DATA_EXPORT.{md,pdf}              this document
                           PIPELINE.md                       running the analysis on a new l
  levels_index.tsv       one row per table below: bait, fraction, what it is, size
  proteins.tsv           11,570 proteins: 30 % identity cluster, held-out block, gene, lengt
  sequences.fasta        the sequences those clusters were built from
  targets.tsv            11,570 x 33, the campaign's derived quantities
  targets_meta.tsv       one row per target: run, bait, kind, description, reliability ceili
  levels/<LEVEL>/
      matrix.tsv         proteins x samples, log2, analysis-ready
      samples.tsv        one row per sample: design fields, BR, cond, n_injections, inj_pos
      proteins.tsv       accession, gene, name, protein group, contaminant flag
      meta.json          source file, the per-sample normalisation offsets, the config used
```

Level	Bait	Fraction	What it is
`GA20_ip`	HSPB1	pull-down	KCl 0 / 75 / 150 mM
`GA22_ip`	HSPB1	pull-down	KCl 0 / 60 / 120 mM x staurosporine 0–10 µM
`GA22_sup`	HSPB1	supernatant	the unbound pool from the same tubes as `GA22_ip`

Level	Bait	Fraction	What it is
`GA24_ip`	HSPB1	pull-down	KCl 0 / 60 / 120 mM x NAD / NADP
`GA24_sup`	HSPB1	supernatant	the unbound pool from the same tubes as `GA24_ip`
`GA33_ip`	DNAJA1, DNAJB11	pull-down	35 / 37 / 43 °C x staurosporine 0 or 10 µM
`GA33_input`	—	pre-IP lysate	the input the GA_33 reactions were drawn from
`FS73_ip`	HSPB1	pull-down	anticonvulsant screen
`FS76_ip`	HSPB1	pull-down	Mg2+ x lithium screen

Rows are UniProt accessions, taken as the first accession of the DIA-NN protein group. Columns of `matrix.tsv` are sample ids that match `samples.tsv["column"]`.

---

2. What was done to the numbers

In order, for the seven DIA-NN runs (`GA22*`, `GA24*`, `GA33*`):

1. **Protein groups keyed by first accession.** Where two groups collapse to the same accession, the better-detected one is kept. The full group string stays in each level's `proteins.tsv`.
2. **Technical MS injections averaged in log2**, over those injections where the protein was detected. `samples.tsv["n_injections"]` records how many went into each sample. This step is why the matrix has 60 columns and not 180.
3. **Median normalisation per sample.** Each sample's offset is the median of its log-ratios to each protein's across-sample mean, computed over proteins detected in >= 90 % of samples. The offsets are kept in `meta.json["offsets"]` so the global shift is recoverable — it was removed, not lost.
4. **Nothing imputed.** A protein not detected in a sample is `NaN`. It is never 0, and it was never filled with a floor, a half-minimum or a group mean.

`GA20_ip` was already reduced to replicate blocks before this repository; it is exported at that level and its normalisation was applied upstream. `FS73_ip` and

`FS76_ip` arrive as proteoDA workbooks, already log2 and normalised upstream, and are passed through unchanged.

Why median and not something stronger. Run medians in GA_33 rise about 1 log2 unit from 35 to 43 °C. That is either loading or a real global increase in binding, and these data cannot tell the two apart. Median normalisation removes a constant per run, so it leaves comparisons across proteins alone and pushes any global shift into the intercept of a fold change, which a model with an intercept absorbs anyway. Scale and quantile normalisation were both tried (`code/ad04_normalisation.py`, `ad04b`) to check whether conclusions depend on compressing the dynamic range. They do not.

Zeros

There are none, and that is deliberate. DIA-NN reports a missing protein as empty, not as zero, and a zero in log2 space would mean an intensity of 1, which is a strong and false claim about a protein that was simply not seen. Missingness here is **not random** — it tracks abundance, and in the pull-downs it also tracks binding. Imputing it with any constant will bias every downstream comparison toward that constant.

If you need complete cases, filter on detection rate (`matrix.notna().mean(axis=1)`) and say what you filtered. The campaign required a protein to have at least 4 of 6 real values in both groups of a contrast.

---

3. Replicates: the part that matters most

`BR` is an independent biological replicate. `TR` is a technical MS injection of the same material, and it has already been averaged away. The columns you see are biological replicates.

This is the single easiest way to get a wrong answer from these tables. GA_33 has 72 samples drawn from 6 biological replicates per condition; if you treat the 1–2 injections behind each as separate observations you roughly double n, and every standard error and p-value comes out too small. The injections measure the mass spectrometer, not the biology.

Level	Conditions	Biological replicates per condition	Injections per replicate
`GA20_ip`	3	5	2
`GA22_ip`	12	5	3–4
`GA22_sup`	12	4–5	1
`GA24_ip`	15	5	2–3

Level	Conditions	Biological replicates per condition	Injections per replicate
`GA24_sup`	15	4–5	1
`GA33_ip`	12	6	1–2
`GA33_input`	12	**1**	3
`FS73_ip`	7	4–6	1
`FS76_ip`	12	3–4	1

Four consequences

The replicates are paired, so pair them. BR 3 at KCl 0 and BR 3 at KCl 120 are the same lysate preparation split between conditions. Fit a replicate block term, or work on within-replicate differences. An unpaired comparison throws away the pairing and loses real power — replicate-to-replicate variation is the largest term in these data.

`GA33_input` has no biological replication. Twelve reactions, one each, three injections apiece. You cannot get an error bar or a reliability estimate for the input from this table. The campaign handled it by building one reference input as the mean over ten of the twelve samples, justified by the input not changing with temperature (`code/ad08_reference_lysate.py`). Two input samples, `21A|0|35` and `24|10|35`, were heavily contaminated and left out of that reference. They are present here, unflagged in the matrix, so you can make your own decision — but do not average all twelve without looking.

Two supernatant blocks failed and are simply absent. `GA22_sup` has no BR 5 at KCl 60, and `GA24_sup` has no BR 5 at KCl 120 — in both cases across every drug level of that salt. So those conditions have 4 replicates where their neighbours have 5. Unbalanced, not missing-at-random within the protein dimension; any code that assumes a rectangular design will quietly mis-align here.

Reliability is already measured, so use it. `targets_meta.tsv` carries a split-half ceiling for each of the 33 targets: replicates split into two halves, each half's estimate correlated against the other across proteins, Spearman-Brown corrected back to the full set. It is the honest upper bound on how well anything can predict that target. Quote accuracy as R^2 divided by the ceiling, not raw. The ceilings range from **0.645** (`GA33_A1_heat37`, a 2 °C step that barely moves anything) to **0.998** (`GA20_ip_base`), and a model scored against the wrong one of those looks three times better or worse than it is.

4. Two run-specific things to know

Injection order drifts in GA_22 and GA_24. Those runs were acquired in blocks, and signal drifts along the acquisition order enough to matter. `samples.tsv["inj_pos"]` is each sample's mean position in the run, scaled 0–1. The campaign's models for the GA_22 / GA_24 pull-downs and partitions carry a quadratic term in it. If you fit anything on these two runs, include `inj_pos` and `inj_pos`^{2} as covariates and check whether your effect survives. It is not optional for the salt contrasts: salt was partly confounded with block.

`cond` is not the same shape in every run, and this is the sharpest edge here. The key is the run's own design fields joined by `|`, in that run's order, so it differs between runs that otherwise look alike:

Run	`cond` format	Zero-salt, drug-free reference
`GA22_ip`, `GA22_sup`	`KCl|stau`	`0|0`
`GA24_ip`, `GA24_sup`	`cof|conc|KCl`	`CTRL|0|0`
`GA33_ip`, `GA33_input`	`bait|stau|temp`	`21A|0|35`, `24|0|35`
`GA20_ip`	`KCl`	`0`
`FS76_ip`	`Mg|Li`	`0.25|0`

Note that GA_22 and GA_24 run their salt in *opposite* positions: `60|0` is KCl 60 in GA_22, while in GA_24 the same salt is `CTRL|0|60`. A selector written for one silently matches nothing on the other, so always read the conditions out of `samples.tsv` rather than assuming the format.

The drug arms are here but were not used. `GA22_ip` contains staurosporine 0.1 / 1 / 10 µM, `GA24_ip` contains NAD and NADP at several concentrations, and `FS73_ip` / `FS76_ip` are drug screens throughout. The campaign used the drug-free arms only, because the drug effects on their own targets are large and specific and would dominate a proteome-wide model. They are exported in full so you can use them; just select on `cond` deliberately rather than averaging across it.

One open question for whoever ran these: the NAD / NADP concentrations in `GA24_ip` are recorded as bare numbers (0.6, 11, 15, 300) with no unit anywhere in the file names. They have been treated as µM. If that is wrong, the `cond` strings are still correct as labels, but any dose-response reading of them is not.

---

5. If you are going to model from sequence

`proteins.tsv` carries `cluster30` (MMseqs2, 30 % identity, 50 % coverage) and `heldout_block` (0–4). **Hold out by block, not at random.** Random

splits put homologues on both sides, and a sequence model then scores well by recognising a close relative it has already seen. In this project that difference is large enough to change conclusions, which is why the blocks ship with the data rather than being left to the user.

`targets.tsv` is already aligned to `proteins.tsv` row for row, so the two join on `accession` without reindexing.

---

6. Checks this export passes

Each baseline target was rebuilt from the exported level tables as a plain mean over biological replicates and correlated against the campaign's own `targets.tsv` column:

Rebuilt from	r
`GA22_ip_base`, `GA24_ip_base`	0.9998, 0.9997
`GA22_sup_base`, `GA24_sup_base`	0.9998, 0.9998
`GA33_A1_ip_base`, `GA33_B11_ip_base`	1.0000, 1.0000
`GA33_input`	0.9995
`GA20_ip_base`	1.0000
`GA22_ip_salt120`, `GA24_ip_salt120`, `GA33_A1_heat43` (contrasts)	0.9988, 0.9995, 1.0000

The residual difference from 1.000 is the campaign's drift covariate and its missing-data handling, which a plain mean does not reproduce. The underlying numbers are the same.